coco script

Tags

Topics

Topic Name	Format	Description	Serialization
robot.h264_video.back	foxglove/CompressedVideo	Back camera, landscape, 720p, 20hz	protobuf
robot.h264_video.front	foxglove/CompressedVideo	Front camera, portrait, 720p, 20hz	protobuf
robot.h264_video.left	foxglove/CompressedVideo	Left camera, landscape, 720p, 20hz	protobuf
robot.h264_video.right	foxglove/CompressedVideo	Right camera, landscape, 720p, 20hz	protobuf
/acu_driver/gps_nav_topic	sensor_msgs/NavSatFix		ros1
/cmd_vel	geometry_msgs/Twist	Command sent to base	ros1
/imu	sensor_msgs/Imu	IMU (xyz, ? forward, down)	ros1
/odom	nav_msgs/Odometry	Wheel odometry	ros1
/speed_fb	See below	Scalar values for velocity	ros1
/object_detection/merged_detections_3d	See below	3D car, pet, and pedestrian detections	ros1

▼ `/speed_fb` msg structure

```
float32 car_speed                # scalar value for vehicle speed
float32 turn_speed               # scalar value for angular velocity
float32 fl_speed                 # front left wheel speed
float32 fr_speed                 # front right wheel speed
float32 rl_speed                 # rear left wheel speed
```

```
float32 rr_speed                  # rear right wheel speed
uint64 speed_timestamp            # timestamp when this was recorded
```

▼ `/object_detection/merged_detections_3d` msg structure

classes

```
 person = 0, car = 1, pet = 2
```

msg

```
object_detection_msgs/Detection3D[] detections
string camera_name
std_msgs/Header header

================================================================================
MSG: object_detection_msgs/Detection3D
object_detection_msgs/BoundingBox3D bbox
uint32 class_id
float32 confidence

================================================================================
MSG: object_detection_msgs/BoundingBox3D
geometry_msgs/Pose center
geometry_msgs/Vector3 size

================================================================================
MSG: geometry_msgs/Pose
# A representation of pose in free space, composed of position and orientation.
Point position
Quaternion orientation

================================================================================
MSG: geometry_msgs/Point
# This contains the position of a point in free space
float64 x
float64 y
float64 z

================================================================================
MSG: geometry_msgs/Quaternion
# This represents an orientation in free space in quaternion form.

float64 x
float64 y
float64 z
float64 w

================================================================================
MSG: geometry_msgs/Vector3
# This represents a vector in free space.
# It is only meant to represent a direction. Therefore, it does not
# make sense to apply a translation to it (e.g., when applying a
# generic rigid transformation to a Vector3, tf2 will only apply the
# rotation). If you want your data to be translatable too, use the
```

```
# geometry_msgs/Point message instead.

float64 x
float64 y
float64 z
================================================================================
MSG: std_msgs/Header
# Standard metadata for higher-level stamped data types.
# This is generally used to communicate timestamped data
# in a particular coordinate frame.
#
# sequence ID: consecutively increasing ID
uint32 seq
#Two-integer timestamp that is expressed as:
# * stamp.sec: seconds (stamp_secs) since epoch (in Python the variable is called 'secs')
# * stamp.nsec: nanoseconds since stamp_secs (in Python the variable is called 'nsecs')
# time-handling sugar is provided by the client library
time stamp
#Frame this data is associated with
string frame_id
```

Camera Parameters

Intrinsics

```python
# camera constants
INPUT_H, INPUT_W = 1080, 1920
INPUT_WH = (INPUT_W, INPUT_H)
FOCAL = 790
K = np.array([
    [FOCAL, 0, INPUT_W / 2.0],
    [0, FOCAL, INPUT_H / 2.0],
    [0, 0, 1]
])
# fisheye distortion parameters
D = np.array([-0.03054249, 0.00974967, -0.00679687, 0.00238449])
```

Extrinsics

Assume front camera is 0.41 meters from the ground.

> ⚠️ Work in Progress

Working with mcap data

1. Data can be visualized with Foxglove Studio
2. Data can be read/manipulated using mcap library

Example Data loading

Mcap is a nice format for distribution and visualization, but it is a little cumbersome to work with while training. Internally, we preprocess the data to make it easier to work with. High level, we just extract video and pose data into the following directory structure:

```
<clip_id>
├── frame_angular_velocities.npy
├── frame_linear_velocities.npy
├── frame_orientations.npy
├── frame_positions.npy
├── frame_speeds.npy
├── frame_times.npy
└── front.mp4
```

▼ python script

```python
import os
import argparse
import subprocess
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor

import tqdm
import numpy as np
from mcap.reader import make_reader
from mcap_protobuf.decoder import DecoderFactory as ProtobufDecoderF
actory
from mcap_ros1.decoder import DecoderFactory as ROS1DecoderFactory


# NOTE(Brad): Emprically measured error on the timestamp from the ca
mera
OFFSET_MS = 150
```

```
def get_aligned_data(input_filepath):
    encoded_frames, frame_times = [], []
    poses, pose_times = [], []
    speeds = []

    # collect all data
    with open(input_filepath, "rb") as stream:
        reader = make_reader(stream, decoder_factories=[ROS1DecoderF
actory(), ProtobufDecoderFactory()])
        found_keyframe = False
        for _, channel, _, d_msg in reader.iter_decoded_messages(top
ics=["robot.h264_video.front", "/odom", "/speed_fb"]):
            # frames
            if channel.topic == "robot.h264_video.front":
                ts = int(d_msg.timestamp.seconds * 1e9 + d_msg.times
tamp.nanos)
                encoded_frame = d_msg.data
                # TODO(Brad): For pyav to decode the first frame, we
need to append the 4 byte start code. not sure why, but we if just d
ump
                # the h264 stream and use ffmpeg to convert it to an
mp4 with: ffmpeg -i test.h264 -c copy test.mp4 we appear to get the
proper
                # frame count
                if not found_keyframe:
                    if not d_msg.keyframe:
                        continue
                    found_keyframe = True

                encoded_frames.append(encoded_frame)
                frame_times.append(ts)
            # Odom
            elif channel.topic == "/odom":
                pose_ts = int(d_msg.header.stamp.to_time() * 1e9)
                pose_times.append(pose_ts)
                poses.append(
                    [
                        d_msg.pose.pose.position.x,
                        d_msg.pose.pose.position.y,
```

```
                        d_msg.pose.pose.position.z,
                        d_msg.pose.pose.orientation.w,
                        d_msg.pose.pose.orientation.x,
                        d_msg.pose.pose.orientation.y,
                        d_msg.pose.pose.orientation.z,
                        d_msg.twist.twist.linear.x,
                        d_msg.twist.twist.linear.y,
                        d_msg.twist.twist.linear.z,
                        d_msg.twist.twist.angular.x,
                        d_msg.twist.twist.angular.y,
                        d_msg.twist.twist.angular.z,
                    ]
                )
            # speeds
            elif channel.topic == "/speed_fb":
                speeds.append([d_msg.speed_timestamp * 1e3, d_msg.ca
r_speed])

    frame_times = np.array(frame_times) - OFFSET_MS * 1e6  # TODO(Br
ad): do we want to do this here? or later
    pose_times = np.array(pose_times)
    poses = np.array(poses)
    speeds = np.array(speeds)

    pose_interp = np.array([np.interp(frame_times, pose_times, poses
[:, i]) for i in range(poses.shape[1])]).T

    # (Brad): I don't know why we did this but linear.z speed is the
scalar speed
    speed_interp = np.interp(frame_times, speeds[:, 0], speeds[:,
1]).astype(np.float32)

    return dict(
        frame_times=frame_times,
        encoded_frames=encoded_frames,
        frame_speeds=speed_interp,
        frame_positions=pose_interp[:, :3],
        frame_orientations=pose_interp[:, 3:7],
        frame_linear_velocities=pose_interp[:, 7:10],
        frame_angular_velocities=pose_interp[:, 10:13],
    )
```

```
def format_mcap(input_fp, output_base, input_base):

    # mirror structure of input data
    input_base = Path(input_base)
    input_fp = Path(input_fp)
    output_suffix = str(input_fp.relative_to(input_base)).replace(".
mcap", "")
    output_dir = os.path.join(output_base, output_suffix)
    data = get_aligned_data(str(input_fp))
    # dump to disk
    os.makedirs(output_dir, exist_ok=True)

    # video
    # TODO(Brad): directly write to mp4
    n_frames = len(data["encoded_frames"])
    output_bytestream = os.path.join(output_dir, "front.h264")
    with open(output_bytestream, "wb") as w:
        for f in data["encoded_frames"]:
            w.write(f)

    # convert to mp4
    output_mp4 = os.path.join(output_dir, "front.mp4")
    result = subprocess.run(
        ["ffmpeg", "-hide_banner", "-loglevel", "quiet", "-i", outpu
t_bytestream, "-c", "copy", "-y", output_mp4], capture_output=True,
text=True
    )
    if result.returncode == 0:
        os.remove(output_bytestream)
    else:
        print(f"[ERROR] failed to convert h264 bytestream to mp4")

    # frame speeds
    keys = ["frame_times", "frame_speeds", "frame_positions", "frame
_orientations", "frame_linear_velocities", "frame_angular_velocitie
s"]
    for key in keys:
        output_path = os.path.join(output_dir, f"{key}.npy")
        assert data[key].shape[0] == n_frames, f"{key=} in {input_fp
```

```
=} does not have correct n_elements: {data[key].shape[0]} != {n_fram
es=}"
        np.save(output_path, data[key])


def format_wrapper(filepath_and_output):
    filepath, input_base, output_base = filepath_and_output
    try:
        format_mcap(filepath, output_base=output_base, input_base=in
put_base)
    except Exception as e:
        print(f"{filepath=} {e}")


def main():
    parser = argparse.ArgumentParser("fmt")
    parser.add_argument("--input", type=str, help="input dir to recu
rsively search for mcap files", required=True)
    parser.add_argument("--output", type=str, help="output dir to pu
t the newly formatted files", required=True)
    parser.add_argument("--workers", type=int, default=os.cpu_count
(), help="number of parallel workers")
    args = parser.parse_args()

    # discover files
    filepaths = []
    for root, dir, files in os.walk(args.input):
        for filename in files:
            if ".mcap" in filename:
                filepaths.append(os.path.join(root, filename))

    print(f"Found {len(filepaths)} files")

    tasks = [(fp, args.input, args.output) for fp in filepaths]
    with ProcessPoolExecutor(max_workers=args.workers) as executor:
        list(tqdm.tqdm(executor.map(format_wrapper, tasks), total=le
n(tasks)))


if __name__ == "__main__":
    main()
```

⚠️ This script will output “frame synched” data by linearly interpolating sensor data to align with when the front camera frame was taken

After converting to this format, you can use torchcodec to easily index the mp4 file like so:

```
from torchcodec.decoders import VideoDecoder

decoder = VideoDecoder("my_video.mp4", device="cpu", dimension_order="N
HWC")

# sequentially read
for frame in decoder:
    # ... logic here
    pass

# or random read
frame_idxs = [0, 10, 30]
decoder = VideoDecoder(fp_video, device="cpu", dimension_order="NHWC")
frames = decoder.get_frames_at(indices=frame_idxs)
```